

1. Using Spring Data JPA Derived Query Methods

java

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long>
{

    // Pattern matching for product name (case-insensitive)
    List<Product> findByNameContainingIgnoreCase(String name);

    // Pattern matching with sorting by price
    List<Product> findByNameContainingIgnoreCase(String name, Sort
sort);

    // Pattern matching with pagination and sorting
    Page<Product> findByNameContainingIgnoreCase(String name, Pageable
pageable);

    // Alternative: Using Like with wildcard
    List<Product> findByNameLikeIgnoreCase(String name);

    // Pattern matching with starting with
    List<Product> findByNameStartingWithIgnoreCase(String prefix);

    // Pattern matching with ending with
    List<Product> findByNameEndingWithIgnoreCase(String suffix);
}
```

2. Using @Query Annotation with Custom Queries

java

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
```

```

import org.springframework.data.repository.query.Param;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long>
{

    // Custom JPQL query with pattern matching
    @Query("SELECT p FROM Product p WHERE LOWER(p.name) LIKE LOWER(CONCAT('%', :name, '%'))")
    List<Product> searchByNamePattern(@Param("name") String name);

    // Pattern matching with sorting by price
    @Query("SELECT p FROM Product p WHERE LOWER(p.name) LIKE LOWER(CONCAT('%', :name, '%'))")
    List<Product> searchByNamePatternSorted(@Param("name") String name,
    Sort sort);

    // Using native SQL query
    @Query(value = "SELECT * FROM products WHERE LOWER(name) LIKE LOWER(CONCAT('%', :name, '%'))",
            nativeQuery = true)
    List<Product> searchByNameNative(@Param("name") String name);
}

```

3. Complete Repository Interface with All Methods

java

```

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository

```

```
public interface ProductRepository extends JpaRepository<Product, Long>
{

    // === PATTERN MATCHING METHODS ===

    // Basic pattern matching (contains)
    List<Product> findByNameContaining(String name);

    // Case-insensitive pattern matching
    List<Product> findByNameContainingIgnoreCase(String name);

    // Pattern matching with wildcard
    List<Product> findByNameLike(String name);

    // Case-insensitive like
    List<Product> findByNameLikeIgnoreCase(String name);

    // Starts with pattern
    List<Product> findByNameStartingWith(String prefix);

    // Ends with pattern
    List<Product> findByNameEndingWith(String suffix);

    // === SORTING BY PRICE METHODS ===

    // Find all sorted by price
    List<Product> findAllOrderByPriceAsc();
    List<Product> findAllOrderByPriceDesc();

    // Pattern matching with sorting by price
    List<Product> findByNameContainingIgnoreCaseOrderByPriceAsc(String
name);
    List<Product> findByNameContainingIgnoreCaseOrderByPriceDesc(String
name);

    // Pattern matching with dynamic sorting
    List<Product> findByNameContainingIgnoreCase(String name, Sort
sort);

    // === PAGINATION METHODS ===
```

```

        // Pattern matching with pagination
        Page<Product> findByNameContainingIgnoreCase(String name, Pageable
pageable);

        // Pattern matching with custom sort
        Page<Product> findByNameContainingIgnoreCase(String name, Sort sort,
Pageable pageable);

        // === CUSTOM QUERIES ===

        // Custom JPQL with pattern matching and sorting
        @Query("SELECT p FROM Product p WHERE LOWER(p.name) LIKE
LOWER(CONCAT('%', :name, '%'))")
        List<Product> searchProducts(@Param("name") String name);

        // Custom JPQL with dynamic sorting
        @Query("SELECT p FROM Product p WHERE LOWER(p.name) LIKE
LOWER(CONCAT('%', :name, '%'))")
        List<Product> searchProductsSorted(@Param("name") String name, Sort
sort);

        // Native query with pattern matching
        @Query(value = "SELECT * FROM products WHERE name ILIKE CONCAT('%',
:name, '%') ORDER BY price",
                nativeQuery = true)
        List<Product> searchProductsNative(@Param("name") String name);
    }

```

4. Service Layer Usage Example

java

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Sort;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {

```

```
@Autowired
private ProductRepository productRepository;

// Search products by name pattern
public List<Product> searchProductsByName(String name) {
    return productRepository.findByNameContainingIgnoreCase(name);
}

// Search and sort by price ascending
public List<Product> searchProductsByNameSortedAsc(String name) {
    return
productRepository.findByNameContainingIgnoreCaseOrderByPriceAsc(name);
}

// Search and sort by price descending
public List<Product> searchProductsByNameSortedDesc(String name) {
    return
productRepository.findByNameContainingIgnoreCaseOrderByPriceDesc(name);
}

// Search with dynamic sorting
public List<Product> searchProductsWithSorting(String name, String
sortBy, boolean ascending) {
    Sort sort = ascending ? Sort.by(sortBy).ascending() :
Sort.by(sortBy).descending();
    return productRepository.findByNameContainingIgnoreCase(name,
sort);
}

// Search with pagination
public List<Product> searchProductsWithPagination(String name, int
page, int size) {
    Pageable pageable = PageRequest.of(page, size,
Sort.by("price").ascending());
    return productRepository.findByNameContainingIgnoreCase(name,
pageable).getContent();
}

// Using custom query
public List<Product> searchWithCustomQuery(String name) {
    return productRepository.searchProducts(name);
}
```

```

    }

    // Search with wildcard pattern
    public List<Product> searchWithWildcard(String namePattern) {
        // Example: namePattern = "%phone%" for containing "phone"
        return productRepository.findByNameLikeIgnoreCase(namePattern);
    }
}

```

5. Controller Usage Example

java

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import org.springframework.data.domain.Sort;
import java.util.List;

@RestController
@RequestMapping("/api/products")
public class ProductController {

    @Autowired
    private ProductService productService;

    @GetMapping("/search")
    public List<Product> searchProducts(
        @RequestParam String name,
        @RequestParam(required = false) String sortBy,
        @RequestParam(required = false, defaultValue = "true")
boolean ascending) {

        if (sortBy != null) {
            return productService.searchProductsWithSorting(name,
sortBy, ascending);
        }
        return productService.searchProductsByName(name);
    }

    @GetMapping("/search/sorted/asc")
    public List<Product> searchProductsSortedAsc(@RequestParam String
name) {

```

```

        return productService.searchProductsByNameSortedAsc(name);
    }

    @GetMapping("/search/sorted/desc")
    public List<Product> searchProductsSortedDesc(@RequestParam String
name) {
        return productService.searchProductsByNameSortedDesc(name);
    }
}

```

6. Product Entity Example

java

```

import javax.persistence.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "name", nullable = false)
    private String name;

    @Column(name = "price")
    private BigDecimal price;

    @Column(name = "description")
    private String description;

    // Constructors
    public Product() {}

    public Product(String name, BigDecimal price) {
        this.name = name;
        this.price = price;
    }
}

```

```
// Getters and Setters
public Long getId() {
    return id;
}

public void setId(Long id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public BigDecimal getPrice() {
    return price;
}

public void setPrice(BigDecimal price) {
    this.price = price;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}
}
```

Key Features Implemented:

1. Pattern Matching Methods:

- `Containing` - matches substring
- `Like` - uses SQL LIKE with wildcards
- `StartingWith` - prefix matching
- `EndingWith` - suffix matching

- Case-insensitive variants

2. Sorting Methods:

- Ascending/descending order
- Dynamic sorting using Sort object
- Combination of pattern matching and sorting

3. Additional Features:

- Pagination support
- Custom JPQL queries
- Native SQL queries
- Service layer implementation
- REST API endpoints